Jhanvi Cars — Complete API / Frontend / Backend E2E Verification Runbook

Commands reconstructed from the project's recent Sep 30–Oct 1 verification sessions, plus the exact Kubernetes checks needed to trace the request path end-to-end.

Scope: Frontend → Nginx / Ingress → API Gateway → microservices → PostgreSQL / Memcached, including authentication, bookings, payments, logs, endpoints, and troubleshooting.

Important: This document preserves commands and architecture evidenced in the project material. Where the material did not establish a specific endpoint, it is marked as a placeholder rather than invented.

1. Jhanvi Cars request architecture

```
Browser / Frontend
      |
      |  fetch("/api...")
      v
Nginx / Kubernetes Ingress
      |
      |  /api -> API Gateway
      v
API Gateway :3000
      |
      +--> Auth Service         :3001
      +--> User Service         :3002
      +--> Driver Service       :3003
      +--> Booking Service      :3004
      +--> Vehicle Service      :3005
      +--> Payment Service      :3006
      +--> Notification Service :3007
              |
              +--> PostgreSQL :5432
              +--> Memcached :11211 (vehicle-service)

Frontend browser rule:
All browser API requests use /api/...; do not expose backend service ports directly.
```

The frontend API client explicitly uses relative /api requests. The gateway owns the backend routing boundary; Kubernetes service discovery uses names such as auth-service:3001 and payment-service:3006. The project's gateway also uses a 5-second proxy timeout.

2. Current service map

Component	Port	Kubernetes Service	Role
API Gateway	3000	api-gateway-service	Single API entry point / routing / auth
Auth	3001	auth-service	Registration, login, session
User	3002	user-service	User operations
Driver	3003	driver-service	Driver operations
Booking	3004	booking-service	Ride lifecycle / bookings
Vehicle	3005	vehicle-service	Vehicle availability / categories
Payment	3006	payment-service	Payments / Razorpay integration
Notification	3007	notification-service	Notifications
Frontend	80	frontend	Vite build served by Nginx
PostgreSQL	5432	postgres	Application database
Memcached	11211	memcached	Vehicle-service cache

3. Local frontend checks used in the project

Start / verify Vite frontend

```
cd ~/jhanvi-dev/frontend
npm run dev -- --host 0.0.0.0 --port 5173
```

Check the API client implementation

```
sed -n '1,220p' src/api/client.ts
```

Check booking API calls

```
sed -n '1,260p' src/api/bookings.ts
```

Check authentication API

```
sed -n '1,220p' src/api/auth.ts
```

Check frontend API references

```
grep -R "/api" src --exclude-dir=node_modules
```

Check Vite variables

```
grep -R "VITE_" . --exclude-dir=node_modules --exclude-dir=dist
```

4. Local backend / API Gateway checks

Build API Gateway

```
cd ~/jhanvi-dev/backend/api-gateway
npm run build
```

Systemd status

```
sudo systemctl status api-gateway
```

Restart

```
sudo systemctl restart api-gateway
```

Follow gateway logs

```
sudo journalctl -u api-gateway -f
```

Recent gateway logs

```
sudo journalctl -u api-gateway --since "10 minutes ago" --no-pager
```

Direct payment health check from the session

```
curl -i http://127.0.0.1:3006/health
```

Expected payment health response

```
HTTP/1.1 200 OK
{"service":"payment-service","status":"UP","database":"UP"}
```

Inspect Nginx payment requests

```
sudo grep 'POST /api/payments' /var/log/nginx/access.log | tail -5
```

Inspect Nginx upstream errors

```
sudo grep -i -E 'payment|3006|upstream|refused|timeout' /var/log/nginx/error.log | tail -30
```

5. Local authenticated end-to-end API check

The project session obtained a JWT from a saved login response and used it to call the booking API through the Vite/Nginx path.

```
TOKEN=$(python3 -c 'import json; print(json.load(open("/tmp/login-vite.json"))["data"]["token"])')

curl -i http://127.0.0.1:5173/api/bookings \
  -H "Authorization: Bearer $TOKEN"
```

The recorded response was HTTP 200 with JSON booking data and an x-request-id header. Do not commit or persist real tokens; use temporary test credentials.

6. Frontend → backend payment flow verified in the project

1. Customer Dashboard calculates the fare.
2. POST /api/bookings creates the booking.
3. POST /api/payments creates the application payment record.
4. Backend creates the Razorpay order.
5. Frontend loads Razorpay Checkout.
6. Razorpay returns payment identifiers/signature.
7. Frontend calls backend verification.
8. Payment status is updated.
9. Booking/payment state is reflected in the UI.

The project source showed this sequence in CustomerDashboard.tsx and the payment API/client code. Exact request bodies should be taken from the current source before executing manual POST tests.

7. Kubernetes baseline checks

Everything in namespace

```
kubectl get all -n jhanvi-cars
```

Pods with node/IP

```
kubectl get pods -n jhanvi-cars -o wide
```

Services

```
kubectl get svc -n jhanvi-cars
```

Ingress

```
kubectl get ingress -n jhanvi-cars
```

PVCs

```
kubectl get pvc -n jhanvi-cars
```

Jobs / migrations

```
kubectl get jobs -n jhanvi-cars
```

Helm release

```
helm status jhanvi-cars -n jhanvi-cars
```

Recent events

```
kubectl get events -n jhanvi-cars --sort-by='.lastTimestamp' | tail -50
```

8. Kubernetes API Gateway checks

Gateway pod

```
kubectl get pods -n jhanvi-cars -l app=api-gateway
```

Deployment

```
kubectl get deployment api-gateway -n jhanvi-cars
```

Rollout

```
kubectl rollout status deployment/api-gateway -n jhanvi-cars
```

Service

```
kubectl get svc api-gateway-service -n jhanvi-cars
```

Endpoints

```
kubectl get endpoints api-gateway-service -n jhanvi-cars
```

Logs

```
kubectl logs deployment/api-gateway -n jhanvi-cars
```

Previous logs after crash

```
kubectl logs deployment/api-gateway -n jhanvi-cars --previous
```

Describe

```
kubectl describe deployment api-gateway -n jhanvi-cars
```

Gateway environment

```
kubectl exec deployment/api-gateway -n jhanvi-cars -- \
  sh -c 'env | grep -E "^(PORT|AUTH_SERVICE_URL|USER_SERVICE_URL|DRIVER_SERVICE_URL|BOOKING_SERVICE_URL|VEH
```

9. Kubernetes direct service checks

Use a temporary curl pod so the test originates inside the cluster and validates Kubernetes DNS + Service routing.

```
kubectl run curl-debug \
  -n jhanvi-cars \
  --rm -it \
  --restart=Never \
  --image=curlimages/curl:8.10.1 \
  -- sh
```

Inside the pod:

```
curl -i http://api-gateway-service:3000/health
curl -i http://auth-service:3001/health
curl -i http://user-service:3002/health
curl -i http://driver-service:3003/health
curl -i http://booking-service:3004/health
curl -i http://vehicle-service:3005/health
curl -i http://payment-service:3006/health
curl -i http://notification-service:3007/health

exit
```

Only treat an endpoint as valid if that service actually implements it. The payment service health response was explicitly confirmed in the project material; other health paths should be checked against each service's current routes.

10. Kubernetes DNS and Service routing

All service endpoints

```
kubectl get endpoints -n jhanvi-cars
```

Individual gateway endpoints

```
kubectl get endpoints api-gateway-service -n jhanvi-cars
```

DNS checks

```
kubectl run dns-debug \
  -n jhanvi-cars \
  --rm -it \
  --restart=Never \
  --image=busybox:1.36 \
  -- nslookup api-gateway-service
```

Check all important DNS names

```
nslookup api-gateway-service
nslookup auth-service
nslookup user-service
nslookup driver-service
nslookup booking-service
nslookup vehicle-service
nslookup payment-service
nslookup notification-service
nslookup postgres
nslookup memcached
```

11. Frontend checks in Kubernetes

Frontend pod

```
kubectl get pods -n jhanvi-cars -l app=frontend
```

Deployment

```
kubectl get deployment frontend -n jhanvi-cars
```

Rollout

```
kubectl rollout status deployment/frontend -n jhanvi-cars
```

Service

```
kubectl get svc frontend -n jhanvi-cars
```

Logs

```
kubectl logs deployment/frontend -n jhanvi-cars
```

Direct frontend HTTP test from cluster

```
kubectl run curl-debug \
  -n jhanvi-cars \
  --rm -it \
  --restart=Never \
  --image=curlimages/curl:8.10.1 \
  -- curl -I http://frontend:80
```

Fetch frontend HTML

```
kubectl run curl-debug \
  -n jhanvi-cars \
  --rm -it \
  --restart=Never \
  --image=curlimages/curl:8.10.1 \
  -- curl -s http://frontend:80 | head
```

12. Ingress / public path checks

Ingress

```
kubectl get ingress -n jhanvi-cars
```

Ingress details

```
kubectl describe ingress -n jhanvi-cars
```

HTTP frontend through ingress

```
curl -I http://<INGRESS-IP>/
```

HTTP API through ingress

```
curl -i http://<INGRESS-IP>/api/health
```

HTTPS frontend

```
curl -I https://<YOUR-DOMAIN>/
```

HTTPS API

```
curl -i https://<YOUR-DOMAIN>/api/health
```

Expected routing for the Helm chart: /api → api-gateway-service:3000 and / → frontend:80.

13. Backend service operational checks

Auth service

```
kubectl get pods -n jhanvi-cars -l app=auth
kubectl get deployment auth -n jhanvi-cars
kubectl rollout status deployment/auth -n jhanvi-cars
```

```
kubectl logs deployment/auth -n jhanvi-cars
kubectl get svc auth-service -n jhanvi-cars
kubectl get endpoints auth-service -n jhanvi-cars
```

User service

```
kubectl get pods -n jhanvi-cars -l app=user
kubectl get deployment user -n jhanvi-cars
kubectl rollout status deployment/user -n jhanvi-cars
kubectl logs deployment/user -n jhanvi-cars
kubectl get svc user-service -n jhanvi-cars
kubectl get endpoints user-service -n jhanvi-cars
```

Driver service

```
kubectl get pods -n jhanvi-cars -l app=driver
kubectl get deployment driver -n jhanvi-cars
kubectl rollout status deployment/driver -n jhanvi-cars
kubectl logs deployment/driver -n jhanvi-cars
kubectl get svc driver-service -n jhanvi-cars
kubectl get endpoints driver-service -n jhanvi-cars
```

Booking service

```
kubectl get pods -n jhanvi-cars -l app=booking
kubectl get deployment booking -n jhanvi-cars
kubectl rollout status deployment/booking -n jhanvi-cars
kubectl logs deployment/booking -n jhanvi-cars
kubectl get svc booking-service -n jhanvi-cars
kubectl get endpoints booking-service -n jhanvi-cars
```

Vehicle service

```
kubectl get pods -n jhanvi-cars -l app=vehicle
kubectl get deployment vehicle -n jhanvi-cars
kubectl rollout status deployment/vehicle -n jhanvi-cars
kubectl logs deployment/vehicle -n jhanvi-cars
kubectl get svc vehicle-service -n jhanvi-cars
kubectl get endpoints vehicle-service -n jhanvi-cars
```

Payment service

```
kubectl get pods -n jhanvi-cars -l app=payment
kubectl get deployment payment -n jhanvi-cars
kubectl rollout status deployment/payment -n jhanvi-cars
kubectl logs deployment/payment -n jhanvi-cars
kubectl get svc payment-service -n jhanvi-cars
kubectl get endpoints payment-service -n jhanvi-cars
```

Notification service

```
kubectl get pods -n jhanvi-cars -l app=notification
kubectl get deployment notification -n jhanvi-cars
kubectl rollout status deployment/notification -n jhanvi-cars
kubectl logs deployment/notification -n jhanvi-cars
kubectl get svc notification-service -n jhanvi-cars
kubectl get endpoints notification-service -n jhanvi-cars
```

14. Authentication checks

```
# Local: obtain a temporary token using the application's login flow.
TOKEN=$(python3 -c 'import json; print(json.load(open("/tmp/login-vite.json"))["data"]["token"])')

# Authenticated API example
curl -i http://127.0.0.1:5173/api/bookings \
  -H "Authorization: Bearer $TOKEN"

# Kubernetes/public equivalent
curl -i http://<INGRESS-IP>/api/<protected-endpoint> \
  -H "Authorization: Bearer $TOKEN"
```

The gateway source uses JWT verification and an INTERNAL_SERVICE_TOKEN for service-to-service trust. Never print secret values into logs or commit them to Git.

15. Booking API verification

The frontend booking API currently exposes these operations in the project source:

```
POST   /api/bookings
GET    /api/bookings
PATCH  /api/bookings/:id/accept
PATCH  /api/bookings/:id/start
PATCH  /api/bookings/:id/complete
```

The project also contains booking cancellation functionality. Use the current src/api/bookings.ts as the source of truth for its exact method/path before manual testing.

```
sed -n '1,260p' ~/jhanvi-dev/frontend/src/api/bookings.ts
grep -RniF "Only drivers can accept bookings" ~/jhanvi-dev/frontend/src ~/jhanvi-dev/backend 2>/dev/null
```

16. Payment API verification

```
sed -n '1,300p' ~/jhanvi-dev/backend/payment-service/src/controllers/paymentController.ts
sed -n '1,260p' ~/jhanvi-dev/frontend/src/api/payments.ts
sed -n '1,240p' ~/jhanvi-dev/frontend/src/utils/razorpay.ts
sed -n '1,240p' ~/jhanvi-dev/backend/payment-service/src/clients/razorpay.client.ts
```

Known direct health result from the project session: payment-service returned HTTP 200 and reported both service and database as UP.

17. PostgreSQL checks

Postgres pod

```
kubectl get pod postgres-0 -n jhanvi-cars
```

StatefulSet

```
kubectl get statefulset postgres -n jhanvi-cars
```

PVC

```
kubectl get pvc -n jhanvi-cars
```

Postgres service

```
kubectl get svc postgres -n jhanvi-cars
```

Postgres endpoints

```
kubectl get endpoints postgres -n jhanvi-cars
```

Postgres logs

```
kubectl logs postgres-0 -n jhanvi-cars
```

Connect

```
kubectl exec -it postgres-0 -n jhanvi-cars -- \
  psql -U jhanvi_admin -d jhanvi_cars
```

Schema checks inside psql

```
\dn
```

```
\du
\dt auth_schema.*
\dt user_schema.*
\dt driver_schema.*
\dt booking_schema.*
\dt vehicle_schema.*
\dt payment_schema.*
\dt notification_schema.*
\q
```

18. Memcached checks

Pod

```
kubectl get pod -l app=memcached -n jhanvi-cars
```

Service

```
kubectl get svc memcached -n jhanvi-cars
```

Logs

```
kubectl logs deployment/memcached -n jhanvi-cars
```

DNS

```
kubectl run dns-debug -n jhanvi-cars --rm -it --restart=Never --image=busybox:1.36 -- nslookup memcached
```

19. Migration checks — important current issue

During the Kubernetes rollout, the migration Jobs for several services failed. The recorded user-service migration failure was not a database connectivity error; Knex failed while loading knexfile.ts because the production image did not have a working TypeScript runtime loader.

```
kubectl get jobs -n jhanvi-cars
kubectl describe job user-migration -n jhanvi-cars
kubectl get events -n jhanvi-cars --sort-by='.lastTimestamp' | tail -30

# Debug pod used during the investigation:
cat > /tmp/user-migration-debug.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: user-migration-debug
  namespace: jhanvi-cars
spec:
  restartPolicy: Never
  containers:
    - name: user-migration
      image: jhanviadmin/jhanvi-cars-user-service:1.0
      command: ["npm", "run", "migrate"]
      envFrom:
        - configMapRef:
            name: jhanvi-cars-config
        - secretRef:
            name: jhanvi-cars-secrets
      env:
        - name: DB_SCHEMA
          value: "user_schema"
EOF

kubectl apply -f /tmp/user-migration-debug.yaml
kubectl logs user-migration-debug -n jhanvi-cars
```

20. CrashLoopBackOff / 502 / 503 troubleshooting

```
# Pod
kubectl get pods -n jhanvi-cars

# Current logs
kubectl logs <POD> -n jhanvi-cars

# Previous crashed container
kubectl logs <POD> -n jhanvi-cars --previous

# Detailed pod state
kubectl describe pod <POD> -n jhanvi-cars

# Service
kubectl get svc <SERVICE> -n jhanvi-cars

# Endpoints
kubectl get endpoints <SERVICE> -n jhanvi-cars

# Gateway direct test
kubectl run curl-debug -n jhanvi-cars --rm -it --restart=Never \
  --image=curlimages/curl:8.10.1 \
  -- curl -i http://api-gateway-service:3000/health

# Events
kubectl get events -n jhanvi-cars --sort-by='.lastTimestamp' | tail -50
```

A 503 in the earlier local session was traced by checking Nginx access/error logs and then confirming payment-service health directly on port 3006. The diagnostic pattern is: identify the failed layer, then test the next layer down.

21. Nginx checks for the local environment

Validate config

```
sudo nginx -t
```

Reload

```
sudo systemctl reload nginx
```

Check status

```
sudo systemctl status nginx
```

Recent access logs

```
sudo tail -50 /var/log/nginx/access.log
```

Recent error logs

```
sudo tail -50 /var/log/nginx/error.log
```

Search upstream failures

```
sudo grep -i -E 'upstream|refused|timeout|502|503' /var/log/nginx/error.log | tail -50
```

22. Final E2E verification sequence

```
STEP 1  kubectl get pods -n jhanvi-cars
STEP 2  kubectl get svc -n jhanvi-cars
STEP 3  kubectl get endpoints -n jhanvi-cars
STEP 4  kubectl get ingress -n jhanvi-cars
STEP 5  curl frontend directly
STEP 6  curl API Gateway /health
STEP 7  curl each backend service from inside cluster
STEP 8  verify DNS resolution
STEP 9  test /api through Ingress
STEP 10 login and obtain temporary JWT
STEP 11 test authenticated /api/bookings
STEP 12 test vehicle availability
STEP 13 create booking
STEP 14 create payment
STEP 15 verify Razorpay/payment flow
STEP 16 verify booking lifecycle
STEP 17 verify PostgreSQL records
STEP 18 verify Memcached / vehicle cache
STEP 19 inspect logs for request ID/errors
STEP 20 inspect migration Jobs and events
```

23. Quick command sheet

```
kubectl get all -n jhanvi-cars
kubectl get pods -n jhanvi-cars -o wide
kubectl get svc -n jhanvi-cars
kubectl get endpoints -n jhanvi-cars
kubectl get ingress -n jhanvi-cars
kubectl get pvc -n jhanvi-cars
kubectl get jobs -n jhanvi-cars
kubectl get events -n jhanvi-cars --sort-by='.lastTimestamp'

kubectl logs deployment/api-gateway -n jhanvi-cars
kubectl logs deployment/frontend -n jhanvi-cars
kubectl describe deployment api-gateway -n jhanvi-cars
kubectl rollout status deployment/api-gateway -n jhanvi-cars
kubectl rollout status deployment/frontend -n jhanvi-cars

helm status jhanvi-cars -n jhanvi-cars

curl -i http://127.0.0.1:3006/health
curl -i http://127.0.0.1:5173/api/bookings -H "Authorization: Bearer $TOKEN"

sudo nginx -t
sudo tail -50 /var/log/nginx/access.log
sudo tail -50 /var/log/nginx/error.log
```

End of runbook — Jhanvi Cars API / Frontend / Backend E2E checks